CRISIS PREDICTION

MACHINE LEARNING PROJECT

CONTEXTO

En un entorno global inestable e incierto, **entender las señales que preceden ciertos eventos es esencial**. Los datos económicos de los países esconden patrones que pueden anticipar momentos de inestabilidad. Este proyecto convierte esa información en una herramienta capaz de **prever riesgos** antes de que estallen.

- **Datos** → se utilizan datos históricos de distintos países para entrenar un modelo de *Machine Learning* capaz de indicar si un país podría entrar en crisis al año siguiente.

- **Análisis** → basado en indicadores macroeconómicos del Banco Mundial y en registros de crisis financieras recopilados por Laeven & Valencia (IMF).

- **Objetivo** → construir una herramienta que permita detectar señales prematuras de vulnerabilidad económica y mejorar la capacidad de anticipación ante posibles episodios de inestabilidad.

VARIABLES DEL DATASET

Sistema Financiero

Variables que describen la salud del sistema bancario y la liquidez. Capturan apalancamiento, riesgo bancario, liquidez y coste del dinero

Deuda y Sostenibilidad

Variables que miden la carga financiera del país. Permiten evaluar si un país puede pagar su deuda o está cerca de un problema de refinanciación

Inflación

Capturan tensiones de precios y posibles desequilibrios macroeconómicos.

Sector Externo

Indicadores que miden la relación del país con el resto del mundo. Detectan vulnerabilidad externa, dependencia de divisas y riesgo cambiario.

Crecimiento Económico

Indicadores del ciclo económico. Ayudan a entender si la economía está expandiéndose o contrayéndose.

Mercado Laboral

Refleja la situación del empleo y la salud económica general

Target

crisis_target

- **1 → El país entra en crisis bancaria en el año siguiente**
- **0 → No hay crisis**

Es la variable que predice el modelo

EDA (ANÁLISIS EXPLORATORIO DE DATOS)

El **EDA** se ha elaborado dividiendo el *train* por los bloques descritos anteriormente para facilitar el análisis, realizando en cada uno:

VISUALIZACIÓN DE HISTOGRAMAS	muchas variables presentan distribuciones muy sesgadas, típico en datos macroeconómicos
BOXPLOTS	muchos outliers, sobre todo en la clase 0. No son errores, sino diferencias reales entre economías
DISTRIBUCIÓN DEL TARGET	proporción entre [Años con crisis **(1)**], [Años sin crisis **(0)**], dataset muy desbalanceado y determinante para la selección de features.
PAIRPLOT	revisar diferencias entre clases, algunas regresiones lineales con correlación
HEATMAP	ciertas features con mucha correlación, también determinante para la selección y para confirmar la visualización con el pairplot
SELECTKBEST CON ANOVA	análisis para la selección de features

PREPROCESADO

- Análisis de **nulos**

```python
# PIPELINE

def pipeline(*steps):
    # Crea una función que aplica varios pasos de procesamiento en orden.
    # La salida de cada paso se usa como entrada del siguiente.
    def wrapper(inputs):
        # Función interna del pipeline.
        # Recorre todos los steps definidos y transforma progresivamente el input.
        for step in steps:
            inputs = step(inputs)
        return inputs
    return wrapper

# Contar nulos originales

def step2(df):
    # Genera una lista con las columnas que queremos analizar/trabajar.
    # Excluye columnas base como identificadores, año y target.
    lista_cols = []
    cols_base = ['Country Name','Country Code','year', 'count_null', 'crisis_target']
    for col in df.columns:
        if col not in cols_base:
            lista_cols.append(col)
    return lista_cols

def cols_nulos(df,lista_cols):
    # Cuenta cuántos valores nulos tiene cada fila en las columnas seleccionadas.
    # También crea una variable binaria ('some_null') que indica si una fila tiene al menos un nulo.
    df = df.copy()
    df["count_null"] = df[lista_cols].isna().sum(axis=1)
    df["some_null"]  = (df["count_null"] > 0).astype(int)
    return df


def cols_nulos_wrapper(df):
    # Función puente para usar cols_nulos dentro del pipeline.
    # Primero obtiene las columnas relevantes con step2 y luego calcula count_null y some_null.
    lista_cols = step2(df)
    return cols_nulos(df,lista_cols)
```

```python
# Contar nulos originales

p = pipeline(cols_nulos_wrapper)

train = p(train)
train
```

✓ 0.0s Python

Current account balance (% of GDP)	Deposit interest rate (%)	Domestic credit to private sector (% of GDP)	Domestic credit to private sector by banks (% of GDP)	Exports of goods and services (% of GDP)	Exports of goods and services (current US$)	...	Official exchange rate (LCU per US$, period average)	Real interest rate (%)	Short-term debt (% of total external debt)	Total debt service (% of exports of goods, services and primary income)	Total reserves in months of imports	Trade (% of GDP)	Unemployment, total (% of total labor force) (national estimate)	crisis_target	count_null	some_null
NaN	NaN	NaN	NaN	NaN	1.711111e+08	...	38.692262	NaN	NaN	NaN	NaN	NaN	NaN	0	24	1
NaN	NaN	NaN	NaN	NaN	2.000000e+08	...	38.692262	NaN	NaN	NaN	NaN	NaN	NaN	0	24	1
NaN	NaN	NaN	NaN	NaN	2.355555e+08	...	38.692262	NaN	NaN	NaN	NaN	NaN	NaN	0	24	1
NaN	NaN	NaN	NaN	NaN	2.244444e+08	...	38.692262	NaN	NaN	NaN	NaN	NaN	NaN	0	24	1
NaN	NaN	NaN	NaN	NaN	3.022222e+08	...	38.378709	NaN	NaN	NaN	NaN	NaN	NaN	0	24	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
7.229714	3.895750	30.645818	30.632417	56.673087	3.956047e+10	...	0.384500	-19.465863	NaN	NaN	4.569866	94.761903	4.142	0	4	1
-0.784048	4.428417	40.746945	40.728990	52.790297	2.927438e+10	...	0.384500	43.178137	NaN	NaN	5.824983	91.627935	NaN	0	5	1
7.594764	3.762167	38.211911	38.196154	59.095638	3.840832e+10	...	0.384500	-6.324583	NaN	NaN	5.476732	96.386555	NaN	0	5	1
11.443571	3.011917	36.103046	36.088178	63.917470	4.953446e+10	...	0.384500	-8.079734	NaN	NaN	5.059990	101.641732	NaN	0	5	1
8.990685	2.688167	36.820169	36.805056	62.721855	5.482445e+10	...	0.384500	2.214694	NaN	NaN	4.364174	102.074760	NaN	0	5	1

PREPROCESADO

- Análisis de **nulos**

```python
# Filtrado de países con >50% de nulos

def step1(df):
    # Genera una lista de los paises de la columna Country Name.
    lista_paises = df['Country Name'].drop_duplicates().to_list()
    return lista_paises

def step2(df):
    # Genera una lista con las columnas que queremos analizar/trabajar.
    # Excluye columnas base como identificadores, año y target.
    lista_cols = []
    cols_base = ['Country Name','Country Code','year', 'count_null', 'crisis_pred']
    for col in df.columns:
        if col not in cols_base:
            lista_cols.append(col)
    return lista_cols

def contador_nulos(df, lista_paises, lista_cols,valor):
    # Calcula, para cada país y cada indicador, el porcentaje de valores nulos.
    # Devuelve solo aquellos pares país-indicador cuyo porcentaje de nulos supera el umbral indicado en 'valor'.

    df = df.copy()
    rows = []
    for pais in lista_paises:
        for col in lista_cols:
            porc_nulos_col = round(df.loc[df['Country Name'] == pais,col].isna().mean(),2)
            if porc_nulos_col > valor:
                rows.append((pais,col,porc_nulos_col))



    return pd.DataFrame(rows,columns=['country','indicator','media_null'])

def contador_nulos_wrapper(df,valor=0.70):
    # Función auxiliar para automatizar el filtrado de nulos dentro del pipeline.
    # Obtiene los países y columnas relevantes, calcula los nulos por país e indicador
    # y devuelve tanto el dataframe resumen como la lista de países afectados.
    lista_paises = step1(df)
    lista_cols = step2(df)
    df_null = contador_nulos(df, lista_paises, lista_cols,valor)
    paises_null = df_null['country'].drop_duplicates().to_list()
    return df_null,paises_null
```

```python
p = pipeline(partial(contador_nulos_wrapper,valor=0.50))

df_null, paises_null = p(df)
```

[33] ✓ 4.9s

```python
df_null.sort_values(ascending=False,by='media_null')
```

[34] ✓ 0.0s

	country	indicator	media_null
2525	Yemen, Rep.	Bank nonperforming loans to total gross loans (%)	1.00
2523	World	Total debt service (% of exports of goods, ser...	1.00
2522	World	Short-term debt (% of total external debt)	1.00
2521	World	Real interest rate (%)	1.00
2520	World	Official exchange rate (LCU per US$, period av...	1.00
...	...	...	...
1505	Micronesia, Fed. Sts.	Broad money (% of GDP)	0.51
1508	Micronesia, Fed. Sts.	Domestic credit to private sector (% of GDP)	0.51
1509	Micronesia, Fed. Sts.	Domestic credit to private sector by banks (% ...	0.51
1515	Micronesia, Fed. Sts.	Lending interest rate (%)	0.51
1517	Micronesia, Fed. Sts.	Real interest rate (%)	0.51

2548 rows × 3 columns

PREPROCESADO

- Imputación de **nulos** → **media por país >> global**

```python
# Relleno de nulos:

def step1(df):
    # Genera una lista de los paises de la columna Country Name.
    lista_paises = df['Country Name'].drop_duplicates().to_list()
    return lista_paises

def step2(df):
    # Genera una lista con las columnas que queremos analizar/trabajar.
    # Excluye columnas base como identificadores, año y target.
    lista_cols = []
    cols_base = ['Country Name','Country Code','year', 'count_null', 'crisis_pred']
    for col in df.columns:
        if col not in cols_base:
            lista_cols.append(col)
    return lista_cols



def relleno_nulos_media_pais(df,lista_paises,lista_cols, how = 'mean'):
    # Rellena los valores nulos de cada indicador usando estadísticas calculadas por país.
    # Si 'how' = 'mean', rellena con la media del país.
    # Si 'how' = 'median', rellena con la mediana del país.
    # Si un país no tiene datos para un indicador, usa como fallback la mediana global de esa columna.
    import numpy as np
    from pandas.api.types import is_numeric_dtype
    df = df.copy()
    if how == 'mean':
        for pais in lista_paises:
            for col in lista_cols:
                if is_numeric_dtype(df[col]):
                    media_col_x_pais = round(df.loc[(df['Country Name'] == pais),col].mean(),6)
                    if np.isnan(media_col_x_pais):
                        media_col_x_pais = round(df[col].median(),6)
                    filtro = df['Country Name'] == pais
                    df.loc[filtro, col] = df.loc[filtro, col].fillna(media_col_x_pais)
                else:
                    continue
        print(df.isna().mean())
        return df
    elif how == 'median':
        for pais in lista_paises:
            for col in lista_cols:
                if is_numeric_dtype(df[col]):
                    media_col_x_pais = round(df.loc[(df['Country Name'] == pais),col].median(),6)
                    if np.isnan(media_col_x_pais):
                        media_col_x_pais = round(df[col].median(),6)
                    filtro = df['Country Name'] == pais
                    df.loc[filtro, col] = df.loc[filtro, col].fillna(media_col_x_pais)
                else:
                    continue
        print(df.isna().mean())
        return df

def relleno_nulos_wrapper(df,how = 'mean'):
    # Función auxiliar para aplicar el relleno de nulos de forma automática.
    # Primero identifica países y columnas relevantes y después aplica la imputación
    # usando media o mediana por país, según se indique en 'how'.
    lista_paises = step1(df)
    lista_cols = step2(df)
    return relleno_nulos_media_pais(df,lista_paises,lista_cols, how)
```

```python
p = pipeline(partial(relleno_nulos_wrapper,how = 'mean'))

train = p(train)

train
```

✓ 6.7s

```
Country Name                                          0.0
year                                                  0.0
Bank nonperforming loans to total gross loans (%)     0.0
Broad money (% of GDP)                                0.0
Current account balance (% of GDP)                    0.0
Deposit interest rate (%)                             0.0
Domestic credit to private sector (% of GDP)          0.0
Domestic credit to private sector by banks (% of GDP) 0.0
Exports of goods and services (% of GDP)              0.0
Exports of goods and services (current US$)           0.0
External debt stocks (% of GNI)                       0.0
Foreign direct investment, net inflows (% of GDP)     0.0
GDP growth (annual %)                                 0.0
GDP per capita growth (annual %)                      0.0
Gross fixed capital formation (% of GDP)              0.0
Gross savings (% of GDP)                              0.0
Imports of goods and services (% of GDP)              0.0
Imports of goods and services (current US$)           0.0
Inflation, GDP deflator (annual %)                    0.0
Inflation, consumer prices (annual %)                 0.0
Lending interest rate (%)                             0.0
Net barter terms of trade index (2015 = 100)          0.0
Official exchange rate (LCU per US$, period average)  0.0
Real interest rate (%)                                0.0
Short-term debt (% of total external debt)            0.0
...
crisis_target                                         0.0
count_null                                            0.0
some_null                                             0.0
dtype: float64
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

	Country Name	year	Bank nonperforming loans to total gross loans (%)	Broad money (% of GDP)	Current account balance (% of GDP)	Deposit interest rate (%)	Domestic credit to private sector (% of GDP)
0	Afghanistan	1970	12.934101	33.241193	-15.123911	5.502083	5.498329
1	Afghanistan	1971	12.934101	33.241193	-15.123911	5.502083	5.498329
2	Afghanistan	1972	12.934101	33.241193	-15.123911	5.502083	5.498329
3	Afghanistan	1973	12.934101	33.241193	-15.123911	5.502083	5.498329
4	Afghanistan	1974	12.934101	33.241193	-15.123911	5.502083	5.498329
...	...	...	...	...	...	...	...
9743	Oman	2008	4.292113	28.067236	7.229714	3.895750	30.645818
9744	Oman	2009	4.292113	37.003465	-0.784048	4.428417	40.746945
9745	Oman	2010	4.292113	35.153251	7.594764	3.762167	38.211911
9746	Oman	2011	4.292113	33.072531	11.443571	3.011917	36.103046
9747	Oman	2012	4.292113	32.467848	8.990685	2.688167	36.820169

9748 rows × 32 columns

MODELADO

- Pipeline para el preprocesado y modelado

```python
# PIPELINE PROCESSING

yeo_pipeline = Pipeline([
    ("yeo", PowerTransformer(method="yeo-johnson", standardize=False)),
    ("scaler", RobustScaler())
])

preprocessing_con_scaler = ColumnTransformer(
        [("Process_Numeric_Log", yeo_pipeline, cols_final),
    ], remainder = "passthrough")

# columnas numéricas del modelo
cols_num = cols_final
preprocessing_sin_scaler = ColumnTransformer(
    transformers=[
        ("impute_num", SimpleImputer(strategy="median"), cols_num),
    ],
    remainder="passthrough"
)

pipe_missings = Pipeline([("first_stage", preprocessing_con_scaler)])
```

```python
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold

cv = KFold(n_splits=5, shuffle=False)

results = []

tree_models = {"RandomForest", "XGBoost", "LightGBM"}

for name, model in models_baseline.items():
    if name in tree_models:
        pipe = Pipeline([("preprocess", preprocessing_sin_scaler),
                         ("model", model)])
    else:
        pipe = Pipeline([("preprocess", preprocessing_con_scaler),
                         ("model", model)])

    scores = cross_val_score(pipe, X_train, y_train, cv=cv, groups=groups,
                             scoring="balanced_accuracy", n_jobs=-1)
    results.append({
        "model": name,
        "balanced_accuracy_mean": scores.mean(),
        "balanced_accuracy_std": scores.std()
    })

pd.DataFrame(results)
```

✓ 13.1s

RESULTADOS

	model	balanced_accuracy_mean	balanced_accuracy_std
0	LogisticRegression	0.723674	0.040189
1	RandomForest	0.499947	0.000105
2	XGBoost	0.510819	0.008589
3	LightGBM	0.499737	0.000407

- **Balanced Accuracy** como métrica de evaluación de los **modelos baseline**
- Elección de **Regresión Logística y XGBoost** para optimización

OPTIMIZACIÓN Y EVALUACIÓN

```python
from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier

xgb = XGBClassifier(
    eval_metric="aucpr", #"logloss",  #o aucpr
    random_state=42,
    scale_pos_weight=spw
)

param_grid_xgb = {
    "model__n_estimators": [100, 200, 300],
    "model__max_depth": [3, 4, 5],
    "model__learning_rate": [0.01, 0.05, 0.1],
    "model__subsample": [0.7, 0.9, 1.0],
    "model__colsample_bytree": [0.7, 0.9, 1.0],
    "model__scale_pos_weight": [
        spw * 0.5,
        spw,
        spw * 1.5,
        spw * 2
    ]

}

pipe_xgb = Pipeline([
    ("preprocess", preprocessing_sin_scaler),
    ("model", xgb)
])

grid_xgb = GridSearchCV(
    estimator=pipe_xgb,
    param_grid=param_grid_xgb,
    scoring="balanced_accuracy",
    cv=cv,
    n_jobs=-1
)
#Entrenar le modelo final con el grid para elegir
print("\n=== OPTIMIZANDO XGBOOST ===")
grid_xgb.fit(X_train, y_train) #esto entrena

print("Mejores parámetros:", grid_xgb.best_params_)
print("Mejor balanced accuracy:", grid_xgb.best_score_)
```

✓ 3m 11.4s

```
=== OPTIMIZANDO XGBOOST ===
Mejores parámetros: {'model__colsample_bytree': 1.0, 'model__learning_rate': 0.01, 'model__max_depth': 3, 'model__n_estimators': 300, 'model__scale_pos_weight': 49.62587412587
Mejor balanced accuracy: 0.7460807979024379
```

```python
best_xgb = grid_xgb.best_estimator_
print("XGB Train")
print(classification_report(y_train, best_xgb.predict(X_train)))

print("XGB Test")
print(classification_report(y_test, best_xgb.predict(X_test)))
```

✓ 0.0s

```
XGB Train
              precision    recall  f1-score   support

           0       1.00      0.71      0.83      9462
           1       0.09      0.89      0.16       286

    accuracy                           0.72      9748
   macro avg       0.54      0.80      0.49      9748
weighted avg       0.97      0.72      0.81      9748

XGB Test
              precision    recall  f1-score   support

           0       0.99      0.66      0.79      4055
           1       0.07      0.81      0.13       124

    accuracy                           0.67      4179
   macro avg       0.53      0.74      0.46      4179
weighted avg       0.96      0.67      0.77      4179
```

OPTIMIZACIÓN Y EVALUACIÓN

```python
# REGRESION LOGISTICA

from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

lg_reg = LogisticRegression(random_state=42)

param_grid_lg_reg = [
    {
        "model__penalty": ["l2"],
        "model__C": [1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1, 3e-1, 1, 3, 10, 30, 100],
        "model__solver": ["lbfgs", "newton-cg", "saga"],
        "model__max_iter": [500, 1000, 2000],
        "model__class_weight": [None, "balanced"],
    },
    {
        "model__penalty": ["l1"],
        "model__C": [1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1, 3e-1, 1, 3, 10, 30, 100],
        "model__solver": ["liblinear", "saga"],
        "model__max_iter": [500, 1000, 2000],
        "model__class_weight": [None, "balanced"],
    },
    {
        "model__penalty": ["elasticnet"],
        "model__C": [1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1, 3e-1, 1, 3, 10, 30, 100],
        "model__l1_ratio": [0.05, 0.1, 0.2, 0.35, 0.5, 0.65, 0.8, 0.9, 0.95],
        "model__solver": ["saga"],
        "model__max_iter": [2000, 4000],
        "model__class_weight": [None, "balanced"],
    },
]

pipe_lg = Pipeline([
    ("preprocess", preprocessing_con_scaler),
    ("model", lg_reg)
])

grid_lg = GridSearchCV(
    estimator=pipe_lg,
    param_grid=param_grid_lg_reg,
    scoring="balanced_accuracy",
    cv=cv,
    n_jobs=-1
)

grid_lg.fit(X_train, y_train)

print("Mejores parámetros:", grid_lg.best_params_)
print("Mejor balanced accuracy:", grid_lg.best_score_)
```

✓ 27m 28.3s

```
Mejores parámetros: {'model__C': 0.3, 'model__class_weight': 'balanced', 'model__max_iter': 500, 'model__penalty': 'l2', 'model__solver': 'lbfgs'}
Mejor balanced accuracy: 0.7179122320465107
```

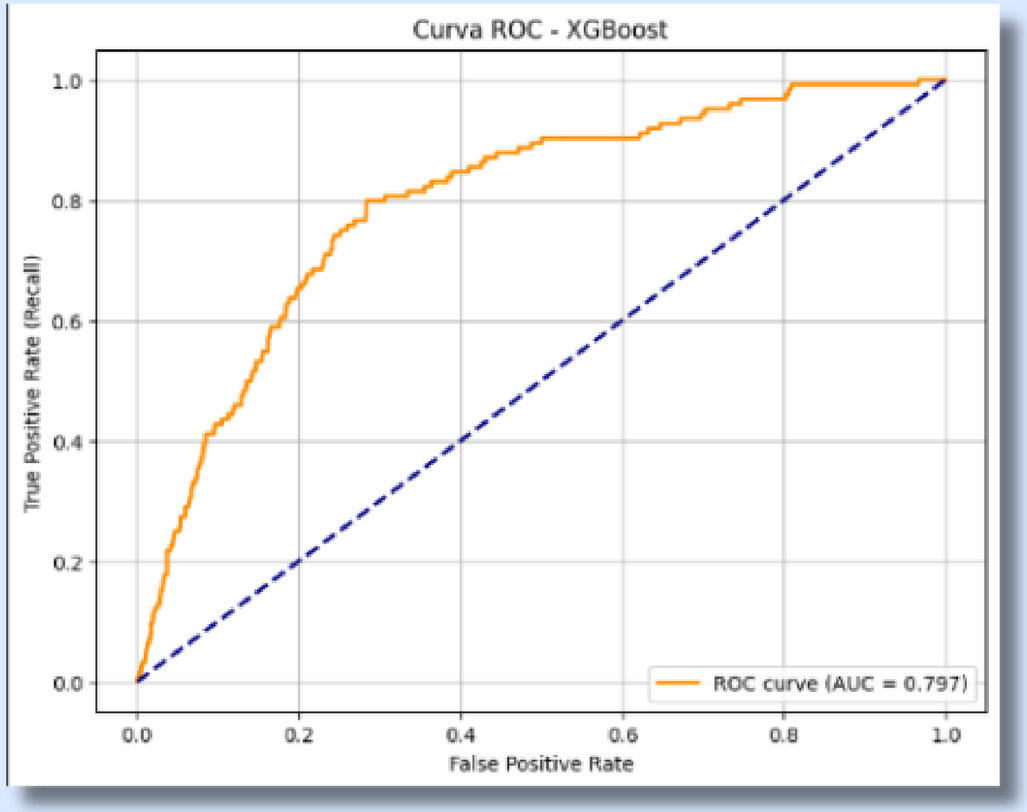
```python
best_lg = grid_lg.best_estimator_
print("LG Train")
print(classification_report(y_train, best_lg.predict(X_train)))
print("LG Test")
print(classification_report(y_test, best_lg.predict(X_test)))
```

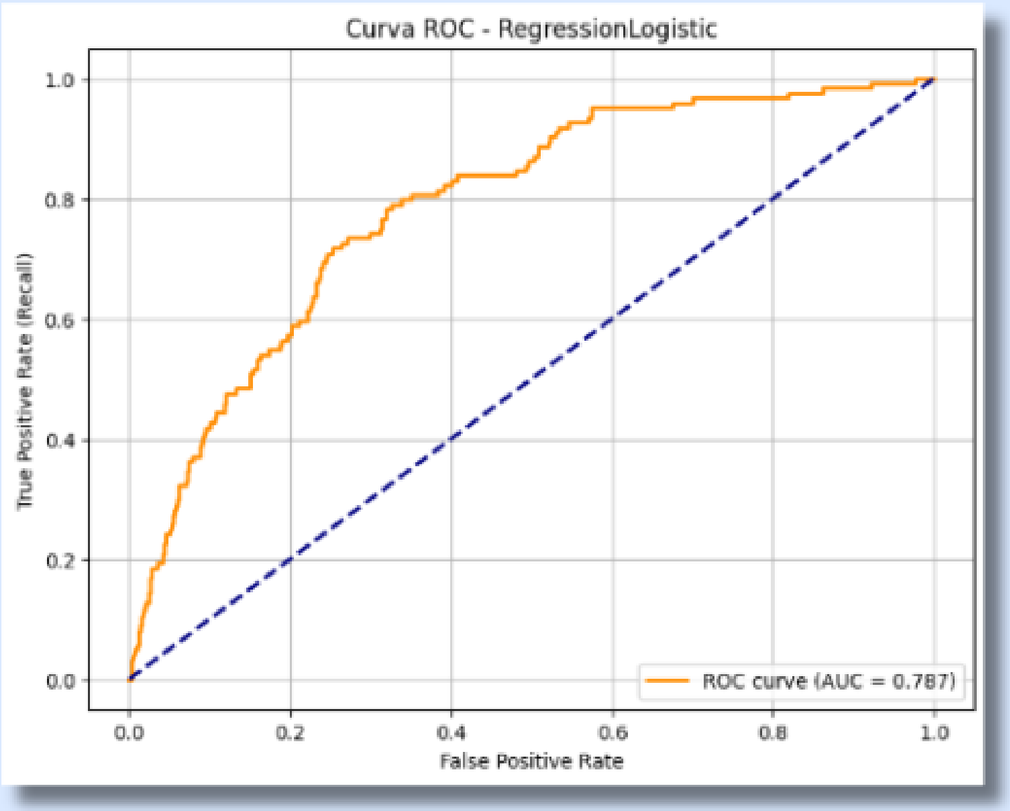
✓ 0.0s

```
LG Train
              precision    recall  f1-score   support

           0       0.99      0.74      0.85      9462
           1       0.08      0.73      0.14       286

    accuracy                           0.74      9748
   macro avg       0.53      0.74      0.50      9748
weighted avg       0.96      0.74      0.83      9748

LG Test
              precision    recall  f1-score   support

           0       0.99      0.72      0.83      4055
           1       0.07      0.73      0.14       124

    accuracy                           0.72      4179
   macro avg       0.53      0.73      0.48      4179
weighted avg       0.96      0.72      0.81      4179
```

COMENTAR CON EQUIPO

EVALUACIÓN


Curva ROC - XGBoost
True Positive Rate (Recall)
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (AUC = 0.797)


Curva ROC - RegressionLogistic
True Positive Rate (Recall)
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (AUC = 0.787)

MODELADO

```python
import numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score

proba = best_xgb.predict_proba(X_test)[:,1]

for t in np.linspace(0.1, 0.9, 17):
    y_pred_thr = (proba >= t).astype(int)
    p = precision_score(y_test, y_pred_thr, zero_division=0)
    r = recall_score(y_test, y_pred_thr, zero_division=0)
    f1 = f1_score(y_test, y_pred_thr, zero_division=0)
    print(f"t={t:.2f}  precision={p:.3f}  recall={r:.3f}  f1={f1:.3f}")
```

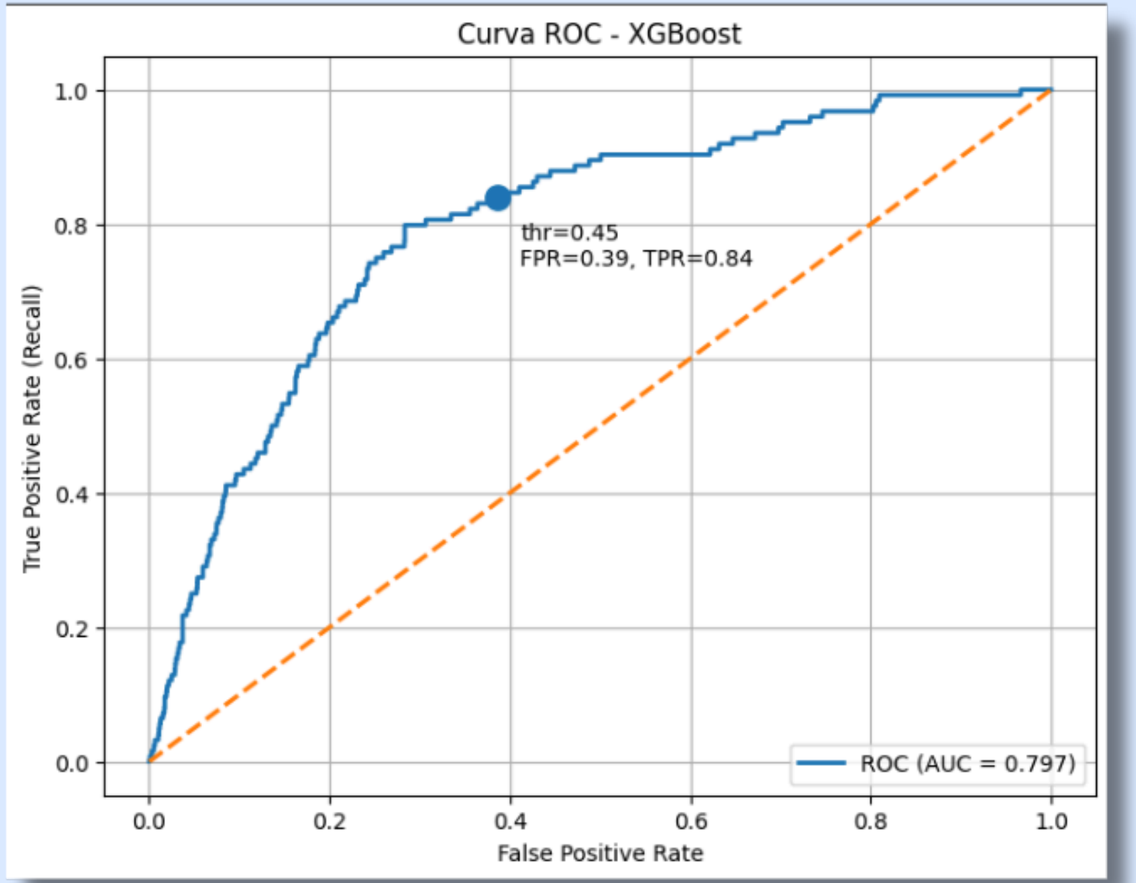
✓ 0.1s

```
t=0.10  precision=0.030  recall=1.000  f1=0.058
t=0.15  precision=0.033  recall=0.992  f1=0.063
t=0.20  precision=0.037  recall=0.984  f1=0.072
t=0.25  precision=0.041  recall=0.960  f1=0.078
t=0.30  precision=0.044  recall=0.919  f1=0.084
t=0.35  precision=0.051  recall=0.903  f1=0.096
t=0.40  precision=0.057  recall=0.871  f1=0.106
t=0.45  precision=0.062  recall=0.847  f1=0.115
t=0.50  precision=0.067  recall=0.806  f1=0.124
t=0.55  precision=0.074  recall=0.782  f1=0.135
t=0.60  precision=0.082  recall=0.750  f1=0.148
t=0.65  precision=0.090  recall=0.685  f1=0.159
t=0.70  precision=0.095  recall=0.597  f1=0.164
t=0.75  precision=0.109  recall=0.532  f1=0.181
t=0.80  precision=0.120  recall=0.395  f1=0.184
t=0.85  precision=0.132  recall=0.234  f1=0.169
t=0.90  precision=0.083  recall=0.008  f1=0.015
```

```python
y_pred_thr = (proba >= 0.45).astype(int)
print(classification_report(y_test,y_pred_thr))
```

✓ 0.0s

```
              precision    recall  f1-score   support

           0       0.99      0.61      0.75      4055
           1       0.06      0.85      0.11       124

    accuracy                           0.61      4179
   macro avg       0.53      0.73      0.43      4179
weighted avg       0.96      0.61      0.73      4179
```



CONCLUSIONES

- La evaluación del modelo mediante precisión, recall y F1-score permitió analizar su rendimiento en ambas clases, diferenciando entre **años con crisis** y **años sin crisis.**
- Dado que el propósito principal es detectar la mayor cantidad posible de crisis reales, se decidió **maximizar la sensibilidad de la clase positiva**, aun cuando ello implique un incremento en las falsas alarmas.
- Este enfoque es coherente con los modelos de predicción de riesgo, donde omitir una crisis potencial resulta mucho más **grave** que generar advertencias.
- El modelo final se caracteriza por una alta sensibilidad ante episodios de crisis y por su utilidad como sistema de **alerta temprana**, aunque a cambio produzca más predicciones de crisis que no llegan a materializarse.
- Esta estrategia se alinea con las prácticas habituales en análisis financiero y económico orientado a la **prevención de eventos críticos.**

GRACIAS POR SU ATENCIÓN